Review package: operation-scoped cancellation (B1–B7)

> **Scope note.** This packet supersedes the earlier "browser lifetime + stop rule S4" packet for the purpose of reviewing the **operation-scoped-cancellation** work. That earlier release (candidate `9a1b931`, base `b676c2d`) is a separate change; its packet is preserved in git history and in the prior patch artifacts. Nothing here touches BM25/aboutness/discovery/counters/scoring/operating-settings, `BATCH_STOP_RULE_FAILED_JOBS` stays unset, and there is no quarantine-recovery override.

Branch	`fix/operation-scoped-cancellation` (pushed to origin)
Base (production)	**`origin/main` = `8c03b07449701a7fb83dc062abbfe6726cf7c208`** — untouched; nothing pushed to it
Release candidate	**`b317833e876332b528759cf9b3ff546d1b20ba39`**
Lineage	`f764c95` (B1 recovery scope + B6 killable pdf-parse subprocess) → **`b317833`** (B2/B3/B7 production-boundary tests + a 20-line B7 test seam in `processor.ts`)
Candidate vs base	`git diff --stat origin/main..b317833` — code changes concentrated in `server/lib/processor.ts` (+623/−132 region) and `server/lib/pipeline.ts` (+253/−… region), plus 6 new test files and supporting fixtures/evidence. Full list in §1.
Gates	No new env switch. The cancellation machinery is active on the isolated-child path that production already runs (`WORKER_ISOLATION_ENABLED=true`). `BATCH_STOP_RULE_FAILED_JOBS` is **not** part of this change and stays unset.
Delivery constraints honoured	Feature branch only; **no push to origin/main**; no merge; no deploy; no paid/Railway runs; no real web/LLM calls.

---

1. What changed (`origin/main..b317833`)

Code:

- **`server/lib/pipeline.ts`** — `runCompanyPdfRecovery` helper (def line 275): arms **one** `OperationScope` for a company's PDF recovery, caps its deadline by `min(PDF_RECOVERY_MAX_BUDGET_MS, remainingMs)`, propagates that scope to **every** fetch call (`fetchFn(origin, slice, { outcomes, scope })`, line 314), stops dispatching once the scope aborts (leaving the untried tail retryable), and caps total attempts

across origins. Called in production at line 1714 with the real `fetchIssuerPdfsWithPrimedSession` as default `fetchFn`.

- **`server/lib/processor.ts`** —
  - **B2:** `acquireProcSlot` queued waiter is cancellable — on abort it withdraws from the local broker queue and consumes no slot (abort listener line 641); grant/abort races hand the slot straight back.
  - **B3:** `runProcessCollect` registers an abort listener that SIGKILLs a hung extraction child at the abort instant (registration 747-750, kill 721-725).
  - **B6:** pdf-parse runs in a **killable subprocess** (`extractViaPdfParseSubprocess` → `pdf-parse-worker.cjs` via `runProcessCollect`), not in-process — a genuine execution boundary, not a 30 MB in-process size cap.
  - **B7:** `processDocument` coalesces concurrent same-URL callers to one fetch **only while the in-flight scope is live**; a caller arriving after that scope aborts starts its own attempt (liveness guard line 2519). The `b317833` commit adds a 20-line `__setProcessDocumentWorkerForTest` seam so this is testable without network.

Tests added (all under `server/lib/`): `company-pdf-recovery-boundary.test.ts` (B1), `cancellable-proc-admission-boundary.test.ts` (B2), `stalled-extraction-boundary.test.ts` (B3), `pdf-parse-subprocess-equivalence.test.ts` (B6), `inflight-coalescing-boundary.test.ts` (B7), plus `operation-scoped-cancellation.test.ts` (scope primitives A/B/C) and the pre-existing `browser-close-order.test.ts` (real-Chromium close/reap ordering).

---

2. Coverage matrix

Full requirement→test→assertion map, with the realism level of each case and the honest gaps, is in **COVERAGE_MATRIX.md**. Headline: 7 test files, 27 subtests, all green on `b317833`; one case (B3) kills a real child process, four cases (browser close/reap) drive a real Chromium child, the rest exercise the real production broker/scope functions in-process.

---

3. Mutation-fail demo per gap (all run this session, applied → observed → reverted clean)

Each guard is shown to be load-bearing: remove it and the named assertion fails. Every mutation was reverted and the tree re-verified clean (`git diff --quiet`, zero `B?-MUT` markers).

Gap	Mutation	Result	Evidence
B1 scope propagation	`fetchFn(origin, slice, { outcomes, scope }) → { outcomes }`	**B1-1 fails**	`AssertionError a scope was passed to the fetch boundary (was DEAD/omit before B1)`; B1-2/B1-3 cascade

B2 queued-waiter abort	remove `sig.addEventListener("abort", onAbort…)` (line 641)	**B2-1 fails**	`'hung' !== 'settled'`; B2-2/3/4 cascade `proc slot leak (active=1)`
B3 stalled-child kill	disable abort registration (747-750)	**B3 fails**	`parent settled promptly after abort (elapsed=60005` — child ran the **full 60 s** timec
B7 coalescing	`if (false && existing …)`	**B7-1 fails**	`2 !== 1` (two fetches)
B7 liveness	`if (existing) return …` (ignore `scope.aborted()`)	**B7-2 fails**	`1 !== 2` (new caller coalesce onto aborted entry)

B7 needs two mutants because the two clauses of the guard (`existing` and `!scope.aborted()`) are independently load-bearing.

---

4. Gate results (`b317833`, this session)

All 7 cancellation test files green (fixtures: local `ciq-int-pg` on 55432, `ciq-int-redis` on 56379; no production `DATABASE_URL`; `PDF_RECOVERY_MAX_BUDGET_MS=600000`):

```
operation-scoped-cancellation        pass 7  fail 0
company-pdf-recovery-boundary        pass 6  fail 0
cancellable-proc-admission-boundary  pass 4  fail 0
inflight-coalescing-boundary         pass 2  fail 0
stalled-extraction-boundary          pass 2  fail 0
pdf-parse-subprocess-equivalence     pass 2  fail 0
browser-close-order (real Chromium)  pass 4  fail 0
```

Total 27 subtests, 0 failures. `browser-close-order` ran against a **real** Chromium (`/usr/local/bin/chromium`), not skipped.

---

5. Normalized tsc comparison

`npx tsc --noEmit`, error sets sorted and path-normalized, candidate vs base (base run in a worktree checked out at `origin/main` with the same `node_modules`):

	errors
base `8c03b07`	62
candidate `b317833`	61
new errors in candidate (`comm -13`)	**0**
errors removed by candidate	1 — `processor.ts(4,22) TS7016: Could not find a declaration file for module 'pdf-parse'`

(`comm -23`)	(eliminated because B6 moved pdf-parse out of the in-process import)

The candidate introduces **zero** new type errors and removes one pre-existing `any`-type warning. All 61 remaining errors are pre-existing vitest/`any` noise identical to base.

---

6. Remaining unbounded / unproven paths (honest boundary)

1. **Active in-page CDP interruption is not instantaneous.** A fetch/navigation/`page.evaluate`/WAF challenge already executing inside Chromium when the scope aborts is **not** torn down mid-operation by the scope machinery. Cancellation is enforced at the next admission gate (B2/B7 refusal), by dispatch suppression (B1-3), and as the hard backstop by the job-isolation SIGKILL of the whole child (observed in `browser-close-order` child-death path). The in-flight operation can run to its own completion/timeout first — `browser-close-order`'s `A_inflight` case demonstrates exactly this by design.
2. **Deadline is bounded around the browser fetch, not threaded into it.** B1 proves the scope is passed and that dispatch stops after the deadline; it does not apply a per-request socket timeout equal to `remainingMs` inside `fetchIssuerPdfsWithPrimedSession`. The effective ceiling on an already-started browser fetch is the pipeline deadline → SIGKILL.
3. **Production recovery scope degenerates to the constant.** The production call omits `remainingMs`, so production recovery uses `PDF_RECOVERY_MAX_BUDGET_MS` (600000) as the scope deadline; the per-company remaining budget is enforced externally by job-isolation SIGKILL. The capping logic (B1-2) is asserted with a test-supplied `remainingMs` and is **not** the narrower cap running in production today.
4. **Specific production stalls unobserved.** The Idemitsu ~18-minute single step and the AXA analysis-prep ~7m26s stall (`TIMEOUT_INVESTIGATION.md`) are not reproduced here and are **not** claimed fixed by this patch.

---

7. Claims kept narrow

- **Candidate only.** `b317833` is a review candidate on a feature branch. Nothing was merged, deployed, or run on Railway/paid providers. `origin/main` is untouched at `8c03b07` (verified by `git ls-remote`).
- **No full scoring fixture.** Per the reviewer's clarification, no end-to-end scoring run is included; the behavioural tests drive the real caller-to-recovery boundary and the real admission/coalescing/subprocess functions directly.
- **What is proven** is the requirement→assertion map in COVERAGE_MATRIX.md plus the §3 mutation demos. **What is not proven** is in §6 and in COVERAGE_MATRIX §6 — read the green suite as exactly that boundary, no wider.

---

8. Reviewer’s 6-point clarification — status

1. **Behavioural recovery boundary (not callee+grep).** Done — `runCompanyPdfRecovery` is real production code (pipeline.ts:275) called in the production pipeline (1714); B1-1..B1-6 assert scope construction, deadline capping, no-dispatch-after-expiry, retryable tail, healthy path, and expiry-through-the-real-admission-boundary. Mutation: scope-drop → B1-1 fail (§3).
2. **Chromium realism + coverage matrix.** Done — COVERAGE_MATRIX.md states per case whether it is real-Chromium-child (R), real-subprocess (S), or broker/unit (U), and lists the active-interruption gap explicitly.
3. **Fixture infra.** Done — `ciq-int-pg` (55432) / `ciq-int-redis` (56379), no production `DATABASE_URL`.
4. **B6 killable execution boundary.** Done — subprocess, not in-process size cap (§1); `pdf-parse-subprocess-equivalence.test.ts` green.
5. **Recovery budget value retained, semantics documented.** `PDF_RECOVERY_MAX_BUDGET_MS=600000` retained; semantic change (company-wide absolute cap; production degeneration to the constant) documented in §6.3 and tested in B1-2/B1-4.
6. **Delivery: feature branch, no origin/main, no deploy/paid runs.** Honoured (§ header + §7).